
Lab W03: Kanto League Roster

CIS 3300, Intermediate Object-Oriented Programming

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Scenario

You are building a small Java program for the Kanto League. The league keeps two separate records for every Pokemon on file: a roster that identifies each Pokemon and its trainer, and a battle log that tracks wins and losses. Your program combines the two into a single ranked leaderboard, along with a standings summary of how each trainer's Pokemon have done overall, and writes both to a report file.

2 Learning Objectives

By the end of this lab you will have practiced:

- creating objects from a provided class using its constructor
- storing objects in an `ArrayList` and reading them back with `get`, `add`, and `size`
- reading a space-separated text file with `Scanner`
- reading a CSV file with the `OpenCSV` library
- using a linear search to find and update an object already in a list
- writing formatted output to a file

3 Provided Files

You are given the two data files below. You are not given `Pokemon.java` as a file; the class is shown below the data files so you can type it in yourself.

3.1 roster.csv

```
1 PokedexID,Name,Type,Trainer
2 006,Charizard,Fire/Flying,Ash Ketchum
3 007,Squirtle,Water,Ash Ketchum
4 018,Pidgeot,Normal/Flying,Gary Oak
5 025,Pikachu,Electric,Ash Ketchum
6 037,Vulpix,Fire,May
7 059,Arcanine,Fire,Gary Oak
8 074,Geodude,Rock/Ground,Brock
9 095,Onix,Rock/Ground,Brock
10 120,Staryu,Water,Misty
11 121,Starmie,Water/Psychic,Misty
12 280,Ralts,Psychic/Fairy,Dawn
13 393,Piplup,Water,Dawn
```

3.2 battles.txt

```
1 025 42 8
2 393 0 0
3 074 15 15
4 121 33 3
5 006 55 30
6 280 12 4
7 018 27 9
8 095 8 22
9 037 5 5
10 120 10 10
11 059 19 6
12 007 20 5
```

3.3 Pokemon.java

```
1 package pokemon;
2
3 public class Pokemon {
4     // Each Pokemon object stores everything we know about one
5     // Pokemon: its identity, which comes from roster.csv, and its
6     // battle record, which comes from battles.txt.
7     private int pokedexId;
8     private String name;
9     private String type;
10    private String trainer;
11    private int wins;
12    private int losses;
13
14    // The constructor runs when this program creates a new Pokemon
15    // object with "new Pokemon(...)". It stores the identity values
16    // and starts wins and losses at 0, since the battle record has
17    // not been read yet at the moment a Pokemon is created from
18    // roster.csv.
19    public Pokemon(int pokedexId, String name, String type, String trainer
20        ) {
```

```
20         this.pokedexId = pokedexId;
21         this.name = name;
22         this.type = type;
23         this.trainer = trainer;
24         this.wins = 0;
25         this.losses = 0;
26     }
27
28     // Getters hand back one field at a time.
29     public int getPokedexId() {
30         return pokedexId;
31     }
32
33     public String getName() {
34         return name;
35     }
36
37     public String getType() {
38         return type;
39     }
40
41     public String getTrainer() {
42         return trainer;
43     }
44
45     public int getWins() {
46         return wins;
47     }
48
49     public int getLosses() {
50         return losses;
51     }
52
53     // Setters fill in the battle record after the object already
54     // exists, which is needed because roster.csv and battles.txt are
55     // read separately.
56     public void setWins(int wins) {
57         this.wins = wins;
58     }
59
60     public void setLosses(int losses) {
61         this.losses = losses;
62     }
63
64     // Win rate is not stored as a field. It is calculated whenever it
65     // is needed, from wins and losses. The if statement avoids
66     // dividing by zero for a Pokemon that has not battled yet.
67     public double getWinRate() {
68         int totalBattles = wins + losses;
69         if (totalBattles == 0) {
70             return 0.0;
71         }
72         return (double) wins / totalBattles;
73     }
```

4 Project Setup

1. Create a new Eclipse project. Right click the `src` folder, choose **New > Package**, and name it `pokemon`. Create `Pokemon.java` in that package and type in the class shown in Provided Files; its first line should read `package pokemon;`.
2. Right click the project name, choose **New > Folder**, and name it `files`. Place `roster.csv` and `battles.txt` in that folder. That is where your program will look for them when you run it from Eclipse.
3. Add OpenCSV and Apache Commons Lang to the project the same way you did in the Racer demo.
4. Create a new class named `KantoLeagueReport.java` in the same `pokemon` package, with a `main` method. This is where you will write the tasks below.

5 Code Commenting Requirements

The following commenting requirements are copied from the course syllabus.

Code Commenting Requirements: You are required to document each source code file in your assignments

- Follow standards in *Core Java, Volume I*, Chapter 4.10, so the javadoc tool can generate HTML documentation.
- Every source code file you write must have general comments (4.10.8 General Comments)
 - You must have at least one sentence describing the class
 - You must include the assignment title
 - You must have a version number (you can start at 1.0)
 - You must have your name as the author
 - See *Core Java, Volume 1*, Listing 3.1 in Chapter 3.2 as a simple example
- Every class must have comments with a short description (4.10.2 Class Comments)
- Every method must have comments including parameters, return, and throw (4.10.3 Method Comments)
- Every static constant field must have comments (4.10.4 Field Comments)
- Other commenting can be done as desired

`Pokemon.java` does not need to be commented to meet these requirements, even though you are typing it in yourself. The requirements above apply to `KantoLeagueReport.java`.

6 Programming Tasks

Complete these five tasks in `KantoLeagueReport.java`, in order. Every roster entry has a matching battle record and every battle record has a matching roster entry, so you do not need to handle a missing match in this lab.

6.1 Task 1: Build the roster

Using `OpenCSV`, read `roster.csv` into an `ArrayList<Pokemon>` named `roster`. For each row, create a new `Pokemon` object with the four values from that row and add it to `roster`.

6.2 Task 2: Fill in battle records

Using `Scanner`, read `battles.txt` one line at a time. Each line has three numbers separated by spaces: a PokedexID, a wins total, and a losses total. For each line, search `roster` for the `Pokemon` whose `getPokedexId()` matches, and call `setWins(...)` and `setLosses(...)` on that object. This is a linear search: walk the list with a for loop, compare, and act on the one that matches.

6.3 Task 3: Build the ranked leaderboard

Create a second list, `ArrayList<Pokemon> leaderboard`, ranked by win rate from highest to lowest. Build it by inserting each `Pokemon` from `roster` directly into its ranked position as you go, using `leaderboard.add(index, pokemon)`, rather than adding everything to the end and sorting afterward.

To find the insertion point for a given `Pokemon`, walk `leaderboard` from the front and stop at the first position whose existing win rate is lower than the new `Pokemon`'s win rate. If a new `Pokemon`'s win rate exactly ties one already in the list, keep going past it. That is, the tie should land after the `Pokemon` already placed, so the one that was inserted first keeps the earlier spot.

6.4 Task 4: Build trainer standings

Create two parallel lists, `ArrayList<String> trainerNames` and `ArrayList<Integer> trainerWins`, both starting empty. Walk through `roster` once. For each `Pokemon`, check whether its trainer is already in `trainerNames` using `indexOf(...)`. If the trainer is not found, add the trainer's name and win total as new entries at the end of both lists. If the trainer is already there, add that `Pokemon`'s wins to the matching entry in `trainerWins` using `get` and `set` at the index you found.

6.5 Task 5: Write the report file

Write a file named `league_report.txt` in your project's `files` folder, with two sections, matching the format in Expected Output below exactly, including the header lines and the blank line between sections. Win rate should be written as a percentage rounded to one decimal place; you can produce that formatting with `String.format("%.1f", winRate * 100)`.

7 Expected Output

Running your finished program against the provided data files should produce a `league_report.txt` that reads exactly as follows. Use this to check your work before submitting.

```
===== Kanto League Leaderboard =====
1. Starmie (Misty) - Win rate: 91.7%
2. Pikachu (Ash Ketchum) - Win rate: 84.0%
```

```

3. Squirtle (Ash Ketchum) - Win rate: 80.0%
4. Arcanine (Gary Oak) - Win rate: 76.0%
5. Pidgeot (Gary Oak) - Win rate: 75.0%
6. Ralts (Dawn) - Win rate: 75.0%
7. Charizard (Ash Ketchum) - Win rate: 64.7%
8. Staryu (Misty) - Win rate: 50.0%
9. Geodude (Brock) - Win rate: 50.0%
10. Vulpix (May) - Win rate: 50.0%
11. Onix (Brock) - Win rate: 26.7%
12. Piplup (Dawn) - Win rate: 0.0%

===== Trainer Standings =====
Ash Ketchum: 117 wins
Gary Oak: 46 wins
Misty: 43 wins
Brock: 23 wins
Dawn: 12 wins
May: 5 wins

```

Notice the tie rule from Task 3 at work: Pidgeot and Ralts both have a 75.0% win rate, and Pidgeot, which was inserted first, kept the earlier spot. The same thing happens across the three-way tie at 50.0%.

8 Grading

This lab is graded using the checklist below. Each item is graded as met or not met, there is no partial credit within a single item, but each item's point value counts toward your total score.

Table 1: Submission and setup, 1 point each

Item	Points
Submitted as a reasonable attempt	1
KantoLeagueReport.java compiles and runs in Eclipse without errors	1
Project and package name match the assignment (pokemon)	1
Project pushed to GitHub, following the walkthrough video	1

Table 2: Code quality, 1 point each

Item	Points
Consistent naming conventions used throughout	1
Consistent formatting and indentation throughout	1
General comment block at the top of KantoLeagueReport.java includes a class description sentence, the assignment title, a version number starting at 1.0, and the author's name, per syllabus 4.10.8	1
Class has a short description comment, per syllabus 4.10.2	1
Methods have comments including parameters, return, and throws, per syllabus 4.10.3	1
Static constant fields have comments, per syllabus 4.10.4	1
No unrequested extra code or output	1

Table 3: Building the roster, 3 points each

Item	Points
Roster correctly built from roster.csv using OpenCSV, one Pokemon object per row	3
Each Pokemon's win and loss totals correctly matched from battles.txt by PokedexID	3

Table 4: Leaderboard, 3 points each

Item	Points
Leaderboard built by inserting each Pokemon into its ranked position with add(index, element), not by sorting a completed list	3
Ties in win rate broken by keeping the earlier-inserted Pokemon in the earlier position	3

Table 5: Trainer standings, 3 points each

Item	Points
trainerNames and trainerWins declared correctly and stay parallel by index	3
Search-or-add pattern correctly updates an existing trainer's total or adds a new one	3

Table 6: Report output, 3 points each

Item	Points
league_report.txt created in the files folder	3
Kanto League Leaderboard section matches Expected Output exactly	3
Trainer Standings section matches Expected Output exactly	3

Total: 38 points.

9 Submission

Push your Eclipse project to your GitHub repository. A short video walkthrough showing how to commit your project directly from Eclipse is available in the Course Information module; follow the steps shown there.

Due date: See course LMS for the due date.

10 Resources

- [Java ArrayList documentation](#) (Oracle) - the full list of ArrayList methods, including get, add, indexOf, and size.

-
- [OpenCSV project page](#) - documentation for the CSVReader class demonstrated in the class lecture.
-

A note on these resources

These links were found with help from an AI system and web search, not from the instructor's own prior use. They look relevant, but none have been personally verified by the instructor. Treat them as a starting point, and let your instructor know if one is broken, off-topic, or wrong.
